ERP-Wesite 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: Next.js 16(App Router) + Supabase 기반 중소기업용 통합 ERP "NEXUS ERP" (재고, 판매, 인사/권한, 설정).
- 목적/대상 사용자: 중소·중견 기업 내부 직원. Master Admin / Inventory Manager / Sales Manager / Staff 4개 역할로 나눠 재고·판매·인사 데이터를 실시간으로 관리 (PRD.md 3.1).
- 폴더 구성 메모
  - 루트 ERP-Wesite/ 자체가 별도 git 저장소(origin = DevOps0906/nexus-erp, 커밋 2개)이며 실제 소스는 모두 여기에 있다.
  - 하위 ERP_Website/ 는 중복 소스가 아니다. DevOps0906/ERP_Website 저장소를 클론한 빈 저장소로 .gitattributes 1개(커밋 "Initial commit")만 들어 있다. nexus-erp 저장소에는 .gitmodules 없이 gitlink로 커밋되어 있고(git ls-files 결과 ERP_Website), 작업 트리의 .gitignore에서는 해당 줄이 빠져 있으며 README.md 는 삭제 상태(미커밋)다. 즉 "nexus-erp 저장소를 ERP_Website 저장소로 옮기려다 멈춘" 상태로 보인다.
  - ERP-Website.pen(Pencil 디자인 파일, 695KB), UI DESIGN.md(Pencil용 프롬프트), CLAUDE.md(구현 지침)가 함께 있다.

영역	기술
프론트	Next.js ^16.2.9 (App Router, Turbopack dev), React 19, Tailwind CSS 4, lucide-react, sonner(토스트), clsx + tailwind-merge
백엔드	Next.js Server Actions(app/actions/*.ts), Route Handler 1개(app/api/auth/callback)
DB	Supabase PostgreSQL, RLS 활성화, 마이그레이션 1개(supabase/migrations/001_initial_schema.sql)
인증	Supabase Auth (이메일/비밀번호 + Google OAuth), @supabase/ssr 쿠키 세션, middleware.ts
실시간	Supabase Realtime (대시보드 1곳에서만 구독)
인프라	배포 설정 없음 (.env.local 에 URL/anon key 2개만)
AI	없음

- 현재 완성도 추정: 약 50%
  - 근거: 화면 5개(대시보드/재고/판매/인사/설정)와 재고 CRUD, 입출고, 판매 등록, 권한 변경 모달은 실제 Supabase 호출로 구현되어 있다. 그러나 (1) RLS 정책이 자기 참조 재귀와 권한 상승 구멍을 갖고 있어 그대로는 정상 동작·보안 둘 다 보장되지 않고, (2) PRD의 "가입 후 Pending → 승인" 흐름, 역할별 메뉴 접근 제어, 판매 추이 차트, 부서 CRUD, Storage 이미지, 반응형 햄버거 메뉴가 없으며, (3) 판매 상태·설정 저장·헤더 검색/버튼 등은 목(mock) 또는 무동작이다.
- 핵심 기능 목록 (구현 여부)

기능 (PRD)	상태	근거
이메일/구글 로그인, 회원가입	부분	app/(auth)/login/page.tsx. 가입 시 이름 미전송, 프로필 미생성
Pending 승인 후 활성화	미구현	로그인/레이아웃 어디에서도 status 검사 없음
역할별 메뉴 접근 제어	미구현	components/layout/Sidebar.tsx:10-16 모든 역할에 동일 메뉴
대시보드 지표 카드 4개	구현	app/(dashboard)/dashboard/page.tsx:8-27
실시간 판매 추이 차트	미구현	DashboardClient.tsx 에 차트 코드 없음
최근 재고 변동 실시간	부분	DashboardClient.tsx:18-30, Realtime publication 설정 없음
위험 재고 테이블	부분	필터 없이 수량 오름차순 5개 (dashboard/page.tsx:29-33)

기능 (PRD)	상태	근거
상품 등록/수정/삭제	구현	app/actions/inventory.ts, InventoryClient.tsx
입출고/조정 이력	부분	비원자적 처리, 조정은 증가만 가능
부서 신설/수정	미구현	조회만 있음 (app/actions/hr.ts:41-46)
임직원 정보(전화, 직급)	미구현	스키마에 컬럼 없음
권한 수정 모달	구현	HRClient.tsx:49-60, app/actions/hr.ts:16-39
Storage 이미지	미구현	image_url 컬럼만 존재

2. 구조 분석

경로	역할
middleware.ts	세션 갱신, /dashboard* 미인증 시 /login 리다이렉트
lib/supabase/server.ts, lib/supabase/client.ts	서버(쿠키)/브라우저 Supabase 클라이언트. 둘 다 anon key 사용, 서비스 롤 키 사용처 없음
app/(auth)/login/page.tsx	로그인/가입 토글 화면, Google OAuth
app/api/auth/callback/route.ts	OAuth 코드 교환 후 profiles 없으면 Staff/Pending 으로 생성
app/(dashboard)/layout.tsx	인증 확인 + 프로필 조회 + 사이드바
app/(dashboard)/dashboard/{page,inventory,sales,hr,settings}	실제 사이드바가 링크하는 라우트 (RSC에서 데이터 조회 후 Client 컴포넌트에 전달)
app/(dashboard)/{inventory,sales,hr,settings}/page.tsx	위 4개 파일과 바이트 단위로 동일한 중복 라우트 (/inventory 등)
app/actions/{inventory,sales,hr,logs}.ts	서버 액션. 쓰기 로직과 일부 조회
components/{dashboard,inventory,sales,hr,settings}/*Client.tsx	페이지별 대형 클라이언트 컴포넌트(161~396줄). 모달, 테이블, 필터를 한 파일에 포함
components/layout/{Sidebar,Header}.tsx	공통 레이아웃
types/index.ts	수기 작성 도메인 타입 (DB 생성 타입 아님)
supabase/migrations/001_initial_schema.sql	5개 테이블, RLS, 샘플 데이터

데이터 흐름: 각 page.tsx(서버 컴포넌트)가 createClient()로 Supabase를 직접 조회해 Client 컴포넌트에 props로 넘기고, 쓰기는 Client 컴포넌트가 app/actions/* 서버 액션을 호출한 뒤 revalidatePath로 갱신한다. 권한은 사실상 RLS에만 의존하며(서버 액션 중 역할 검사는 updateProfilePermission 하나뿐), 대시보드만 브라우저 클라이언트로 Realtime을 구독한다. 조회 로직이 page.tsx 와 app/actions/*(getProducts, getSalesTransactions, getProfiles, getTodaySalesTotal)에 이중으로 존재하고 액션 쪽 조회 함수는 대부분 호출되지 않는다.

3. 발견된 문제점

#	심각도	분류	위치(경로:줄)	내용
1	높음	보안	supabase/migrations/001_initial_schema.sql:76-80	profiles_update_own/profiles_insert_own 에 컬럼 제한·WITH CHECK 가 없어 누구나 자기 role='Master Admin', status='Active' 로 갱신/삽입 가능 (권한 상승)
2	높음	버그	supabase/migrations/001_initial_schema.sql:83-89	profiles 정책 안에서 profiles 를 다시 조회 → PostgreSQL "infinite recursion detected in policy" 오류. 다른 모든 테이블 정책도 profiles 를 서브쿼리하므로 전 테이블 조회가 실패할 수 있음

#	심각도	분류	위치(경로:줄)	내용
3	높음	보안	app/(dashboard)/layout.tsx:8-15, middleware.ts:28	로그인만 확인하고 status(Pending/Suspended)와 role 을 검사하지 않음. Staff 도 /dashboard/hr, /dashboard/settings 진입 가능. PRD의 승인 흐름이 앱 레벨에서 없음
4	높음	버그	app/actions/sales.ts:20-43, app/actions/inventory.ts:40-64	읽기-계산-쓰기 3단계 비원자 처리. 동시 판매 시 재고 음수/중복 차감, 재고 갱신 성공 후 로그 insert 실패 시 이력 유실. 주석은 "원자적"이라 되어 있으나 아님
5	높음	버그	app/(auth)/login/page.tsx:49	이메일 가입 시 name 을 options.data 로 넘기지 않고, 프로필 생성 트리거도 없음. 이메일 가입자는 profiles 행이 없어 HR 목록에 안 보이고 승인도 불가
6	중간	보안	app/actions/inventory.ts:16-90, app/actions/sales.ts:16	서버 액션에 인증/역할 검사 없음. RLS 하나에만 의존(그 RLS가 #1, #2 상태)
7	중간	품질	components/sales/SalesClient.tsx:174-175, app/(dashboard)/dashboard/sales/page.tsx:33	거래 상태(완료/대기/취소)를 인덱스로 가짜 생성, 취소 건수 0 하드코딩. 스키마에 status 컬럼 없음
8	중간	보안	components/settings/SettingsClient.tsx:232-241	실 Supabase 프로젝트 URL 하드코딩, anon key 앞부분 표시. 설정 화면이 역할 검사 없이 모든 사용자에게 노출
9	중간	품질	components/settings/SettingsClient.tsx:50-52, 36-37	"변경사항 저장"은 토스트만 띄우고 아무것도 저장하지 않음. 안전재고 기본값/알림 토글은 로컬 state
10	중간	구조	app/(dashboard)/{hr,inventory,sales,settings}/page.tsx	dashboard/ 하위 4개와 완전히 동일한 중복 라우트. /hr 등은 middleware 보호 대상(/dashboard prefix) 밖
11	중간	버그	app/actions/sales.ts:51-52, app/(dashboard)/dashboard/page.tsx:21, app/(dashboard)/dashboard/sales/page.tsx:7	setHours(0,0,0,0) 이 서버 TZ(배포 시 UTC) 기준이라 "금일 매출"이 KST 09시 기준으로 잘림 (Hospital/School 과 동일 버그)
12	중간	버그	components/dashboard/DashboardClient.tsx:21-26	products 변경마다 window.location.reload(). Realtime INSERT payload 에 조인 products 가 없어 새 로그 상품명이 "—" 로 표시
13	중간	설정	supabase/migrations/001_initial_schema.sql (전체)	supabase_realtime publication 에 테이블 추가 구문이 없어 Realtime 이벤트가 오지 않음
14	중간	버그	app/actions/inventory.ts:47, components/dashboard/DashboardClient.tsx:104-108	ADJUSTMENT 는 항상 +quantity(감소 조정 불가, 입력 min=1). 대시보드는 IN 이 아니면 모두 "출고/-" 로 표시
15	낮음	버그	app/(dashboard)/dashboard/page.tsx:29-33	"위험 재고"가 quantity <= safety_stock 필터 없이 수량 오름차순 5개. 위험 품목이 없어도 정상 품목이 경고 표에 노출
16	낮음	품질	components/layout/Header.tsx:16-18, 33	검색창은 , 알림 벨 장식뿐, 액션 버튼은 어느 페이지도 onAddClick 을 넘기지 않아 무동작 (hr/page.tsx:14 "직원 등록" 포함)
17	낮음	구조	app/actions/hr.ts:38, app/actions/hr.ts:6, app/actions/sales.ts:5, 49	revalidatePath('/dashboard/hr') 만 갱신. getProfiles/getSalesTransactions/getTodaySalesTotal 등 미사용 함수 다수
18	낮음	설정	package.json:9, 31	Next 16 에서 제거된 next lint 스크립트, eslint-config-next 15.3.3 로 메이저 불일치. Next 16 권장 proxy.ts 대신 middleware.ts 사용
19	낮음	설정	ERP_Website/ (gitlink), .gitignore	.gitmodules 없는 gitlink, 미커밋 .gitignore 변경과 README.md 삭제. 상위 platform 저장소에도 미추적 상태로 3중 저장소 구조

#	심각도	분류	위치(경로:줄)	내용
20	낮음	품질	components/layout/Sidebar.tsx:31	사이드바 고정 260px, 반응형 햄버거 메뉴 없음 (PRD 4장)

심각도 높음 상세

1. 권한 상승 (#1): profiles_update_own 은 USING (auth.uid() = id) 만 있고 WITH CHECK 와 컬럼 제한이 없다. 로그인한 아무 사용자(Pending 포함)가 브라우저 콘솔에서 anon key 로 supabase.from('profiles').update({ role: 'Master Admin', status: 'Active' }).eq('id', myId) 를 호출하면 즉시 최고 관리자가 된다. app/actions/hr.ts:28 의 서버측 검사는 이 경로를 막지 못한다. profiles_insert_own 도 동일하게 role/status 를 임의 지정해 삽입할 수 있다.
2. RLS 재귀 (#2): profiles_admin_all 이 profiles 를 서브쿼리하므로 profiles SELECT 시 정책 평가가 자기 자신을 다시 호출한다. Supabase 에서 흔히 발생하는 42P17 오류이며, departments/products/inventory_logs/sales_transactions 정책도 모두 profiles 서브쿼리를 쓰기 때문에 연쇄적으로 실패한다. SECURITY DEFINER 함수(public.current_role(), public.is_active())로 역할 조회를 분리해야 한다.
3. 상태/역할 미검사 (#3): 레이아웃은 user 존재만 확인한다. Pending 사용자가 로그인하면 빈 화면(또는 #2 오류)이 뜰 뿐 "승인 대기" 안내가 없고, Staff 가 인사·설정 화면과 시스템 로그(판매 금액 포함)를 볼 수 있다. 중복 라우트(/hr, /settings 등)는 middleware 매칭에서도 빠진다.
4. 비원자 재고 처리 (#4): createSale 은 수량을 읽고 JS 에서 뺀 값을 update 한 뒤 별도로 insert 한다. 두 요청이 동시에 오면 둘 다 같은 product.quantity 를 읽어 한 건만 차감된다. 또 total_price 를 서버에서 계산하긴 하지만, RLS 상 Sales Manager 는 products UPDATE 권한이 없어(products_write 는 Admin/Inventory Manager 만) 판매 시 재고 차감 자체가 실패한다. PostgreSQL 함수(rpc('record_sale'), rpc('record_inventory_move'))로 UPDATE ... SET quantity = quantity - $1 WHERE quantity >= $1 와 로그 insert 를 한 트랜잭션에서 처리해야 한다.
5. 이메일 가입 프로필 누락 (#5): 코드 주석은 "auth trigger 또는 callback에서 처리"라 하지만 마이그레이션에 트리거가 없고, signUp 에 emailRedirectTo 도 없어 콜백 라우트를 거친다는 보장이 없다. 결과적으로 이메일 가입자는 프로필이 없어 사이드바가 'Admin' 기본값(Sidebar.tsx:69)으로 표시되는 등 오해 소지도 있다.

4. 효율성 개선 제안

- 구조/아키텍처
  - RLS 를 재작성한다: auth_role()/auth_is_active() SECURITY DEFINER 함수 + 정책에서 호출. profiles 자기 수정은 name 등 비권한 컬럼만 허용(트리거로 role/status 변경 차단 또는 컬럼 GRANT 분리). insert 정책은 제거하고 handle_new_user 트리거로 생성.
  - 재고/판매 쓰기를 DB 함수(RPC) 2개로 옮겨 원자성 확보. 서버 액션은 requireRole([...]) 헬퍼 호출 후 RPC 호출만 담당.
  - 중복 라우트 app/(dashboard)/{hr,inventory,sales,settings} 삭제. 역할→허용 경로 매핑을 lib/permissions.ts 에 두고 proxy.ts(Next 16) + 레이아웃에서 이중 검사. 사이드바도 같은 매핑으로 메뉴 필터링.
  - Hospital/School 리뷰 4장과 비교하면 세 프로젝트가 같은 보일러플레이트를 반복한다: @supabase/ssr 서버/브라우저 클라이언트, middleware(proxy) 세션 갱신 + 역할 가드, profiles(role,status) + 가입 트리거, 좌측 사이드바/상단 헤더 레이아웃, 필터+테이블+모달 CRUD 화면, KST 날짜 유틸. 공통 패키지(packages/admin-kit: supabase/{server,client}.ts, requireRole, permissions 팩토리, AppShell(Sidebar+Header, nav 항목 props), DataTable, FormModal, ConfirmDialog, kstDateRange())로 묶으면 세 프로젝트 모두 이득이다. ERP 는 셋 중 가장 작고(소스 약 33개) 역할 4개 구조가 단순해 공통 키트의 첫 적용 대상으로 적합하다.
- 성능

  - 대시보드 지표는 products 전체 행과 profiles 전체 행을 가져와 JS 로 세고 있다(dashboard/page.tsx:14, 22). rpc('dashboard_metrics') 1회 또는 count: 'exact', head: true 로 대체.
  - Realtime 은 window.location.reload() 대신 router.refresh() 또는 payload 기반 부분 갱신. 새 로그는 상품명을 위해 단건 조회하거나 클라이언트가 가진 상품 맵으로 보강.
  - 위험 재고는 quantity <= safety_stock 를 DB 에서 걸러야 하는데 PostgREST 는 컬럼 간 비교가 안 되므로 생성 컬럼(is_low_stock) 또는 뷰로 처리.
- 개발 생산성
  - supabase gen types typescript 로 types/supabase.ts 생성, createServerClient<Database> 적용. 현재 as { name?: string } 캐스팅(HRClient.tsx:39, DashboardClient.tsx:100)이 제거된다.
  - 400줄 가까운 *Client.tsx 에서 모달을 분리(ProductFormModal, InventoryLogModal, NewSaleModal, PermissionModal). 공통 Modal/inputCls 를 한 곳에 둔다.
  - 미사용 조회 액션(getProducts, getProfiles, getSalesTransactions, getTodaySalesTotal) 정리 또는 page.tsx 가 이를 사용하도록 일원화.
  - package.json 의 lint 를 eslint . 로, eslint-config-next 를 16.x 로 맞춘다. 최소 테스트로 RLS 정책(pgTAP 또는 Supabase 로컬 + 역할별 JWT)과 재고 RPC 를 검증.
- 운영/배포
  - 마이그레이션 분리: 001_schema.sql, 002_rls.sql, 003_triggers_rpc.sql, seed.sql. alter publication supabase_realtime add table products, inventory_logs, sales_transactions; 추가.
  - .env.example 추가, SettingsClient 의 하드코딩 URL 은 process.env.NEXT_PUBLIC_SUPABASE_URL 로 대체하고 anon key 표시는 제거.
  - 저장소 정리: nexus-erp 와 ERP_Website 중 하나를 원격으로 확정하고 gitlink 제거, 상위 platform 저장소에서는 서브모듈 또는 무시 처리.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	RLS 전면 재작성: auth_role()/auth_is_active() SECURITY DEFINER 함수 도입, 재귀 제거, profiles 자기 수정 시 role/status 변경 차단(트리거), insert 정책 삭제, 역할별 SELECT 범위(Staff 는 판매/로그 조회 제한) 정의 → 002_rls.sql	P0	M	Fable 5.1	권한 상승과 전면 장애가 걸린 보안 핵심. 실수 비용이 가장 큼
2	handle_new_user 트리거로 가입 즉시 profiles(Staff, Pending) 생성, signUp 에 options.data.full_name 와 emailRedirectTo 추가, 콜백 라우트의 insert 제거	P0	S	Opus 5.5	인증 흐름이지만 범위가 명확하고 #1 설계를 따르면 됨
3	재고/판매 RPC(record_sale, record_inventory_move)를 PL/pgSQL 로 작성(조건부 UPDATE + 로그 insert 단일 트랜잭션, ADJUSTMENT 음수 허용), 서버 액션을 RPC 호출로 교체	P0	M	Fable 5.1	재고·매출 데이터 무결성과 동시성, RLS 권한(판매자의 재고 차감)과 얽힘
4	lib/permissions.ts(역할→경로 매핑) + requireRole() 헬퍼 작성, 모든 서버 액션 첫 줄에 적용, 레이아웃에서 Pending/Suspended 는 /pending 안내 페이지로 보냄	P0	M	Opus 5.5	2~5개 파일에 걸친 명확한 인가 로직
5	middleware.ts → proxy.ts 전환, 보호 대상에 전체 대시보드 그룹 포함, 역할별 리다이렉트. 중복 라우트 app/(dashboard)/{hr,inventory,sales,settings} 4개 삭제	P1	S	Opus 5.5	가드 누락이 보안과 직결되어 Sonnet 보다 한 단계 위
6	사이드바 메뉴를 permissions.ts 로 필터링하고 Sidebar.tsx:69 의 'Admin' 기본값 제거, 태블릿 이하 햄버거 메뉴 구현	P1	S	Sonnet 5.5	UI 반복 작업

#	작업	우선순위	예상규모	권장 모델	선정 이유
7	KST 기준 kstDayRange() 유틸 작성 후 sales.ts:51, dashboard/page.tsx:21, dashboard/sales/page.tsx:7 교체 (Hospital/School 과 공유 가능하게)	P1	S	Sonnet 5.5	단순 버그, 패턴 명확
8	마이그레이션에 Realtime publication 추가, DashboardClient 의 window.location.reload() 를 router.refresh() 로 바꾸고 새 로그 상품명 보강, ADJUSTMENT 표시 수정, 위험 재고를 is_low_stock 생성 컬럼으로 필터	P1	M	Opus 5.5	DB + 클라이언트 동시 수정, 중간 난이도
9	판매 상태 실제화: sales_transactions.status('completed','pending','cancelled') 컬럼 추가, 취소 액션(재고 복원 포함 RPC), SalesClient.tsx:174 가짜 상태와 cancelCount={0} 제거	P1	M	Opus 5.5	기능 구현 + 재고 복원 로직
10	대시보드 판매 추이 차트(시간대별 금액, SVG 또는 경량 차트) 및 rpc('dashboard_metrics') 집계로 전체 행 조회 제거	P2	M	Opus 5.5	명확한 중간 규모 기능
11	부서 CRUD(생성/이름 수정) 서버 액션과 UI, profiles 에 phone, position 컬럼 추가 및 HR 목록 표시	P2	M	Opus 5.5	PRD 3.4 미구현분, 정의가 명확
12	설정 화면: 하드코딩 URL/anon key 표시 제거, Master Admin 전용 가드, 저장 기능은 settings 테이블로 구현하거나 버튼 제거. 헤더 검색/벨/무동작 버튼 제거 또는 연결	P2	S	Sonnet 5.5	UI/문구 정리 위주
13	supabase gen types 로 DB 타입 생성 후 캐스팅 제거, 대형 *Client.tsx 에서 모달 분리, 미사용 조회 액션 삭제	P2	M	Sonnet 5.5	반복적 리팩토링·타입 정리
14	package.json lint 스크립트/eslint-config-next 16.x 정리, .env.example 추가, ERP_Website gitlink 정리와 원격 저장소 확정, README 복구	P2	S	Sonnet 5.5	설정·문서 정리

6. 종합 평가

- 강점
  1. 구조가 작고 일관적이다: RSC 조회 → Client 컴포넌트 → Server Action → revalidatePath 패턴이 모든 화면에 동일하게 적용되어 수정 범위를 예측하기 쉽다.
  2. 서비스 롤 키를 어디에서도 쓰지 않고 anon key + 쿠키 세션 + RLS 구조를 택했다. 방향 자체는 올바르며 정책만 고치면 된다.
  3. PRD 와 디자인 토큰(Blue #0047AB / Orange #FF8C00, rounded-lg)이 화면에 충실히 반영되어 UI 완성도는 높은 편이고, 재고 CRUD·입출고 모달·권한 모달이 실제 DB 와 연결되어 있다.
- 약점
  1. RLS 가 재귀 오류와 자기 권한 상승 구멍을 동시에 갖고 있어, "RLS 기반 RBAC" 라는 PRD 핵심 가치가 현재 성립하지 않는다.
  2. 재고·판매 쓰기가 비원자적이며 판매 역할은 재고 차감 권한조차 없어 실제 운영 시 데이터가 어긋난다.
  3. 앱 레벨 인가(상태/역할 검사, 메뉴 필터)가 거의 없고, 판매 상태·설정 저장·헤더 기능 등 목/무동작 UI 와 중복 라우트가 섞여 있다.
- 모델별 비율 (14개 작업 기준): Fable 5.1 2개(14%) / Opus 5.5 7개(50%) / Sonnet 5.5 5개(36%)
- 권장 진행 순서

1. 1단계 (보안·무결성 확보): 작업 1 → 2 → 3 → 4. RLS 재작성과 RPC 를 먼저 끝낸 뒤 서버 액션을 그 위에 맞춘다. 로컬 Supabase 에서 역할별 계정으로 권한 상승·동시 판매 시나리오를 검증.
2. 2단계 (라우팅·실시간·데이터 정확성): 작업 5 → 6 → 7 → 8 → 9. 이 시점에 Hospital/School 과 공통 `permissions`/`AppShell`/`kstDayRange` 를 추출할지 결정.
3. 3단계 (PRD 잔여 기능과 정리): 작업 10 → 11 → 12 → 13 → 14.

- 주 담당 모델: Opus 5.5. 이 프로젝트는 소스 약 33개의 작은 코드베이스이고 대부분의 남은 일(인가 헬퍼, 라우트 가드, 판매 상태, 차트, 부서 CRUD, Realtime 정리)이 범위가 명확한 2~5개 파일 단위 기능 작업이라 Opus 5.5 가 비용 대비 가장 적합하다. 다만 RLS 재작성(작업 1)과 재고/판매 RPC(작업 3)는 권한 상승과 재고·매출 무결성이 걸린 고위험 작업이므로 이 두 건만 Fable 5.1 에 맡기고, 결과물(정책·함수 설계)을 Opus 5.5 가 이어받아 앱 코드에 반영하는 방식을 권장한다. UI·설정·타입 정리는 Sonnet 5.5 로 내려 비용을 줄인다.